ZeroFloat: Fully Integer Training of Language Models up to 1B Parameters

Muhammed Ökmen
Independent Researcher
okmen1306@gmail.com ORCID: 0009-0002-9709-966X

October 2026 Version 2 (v1 DOI: 10.5281/zenodo.23450386)

Abstract

We train decoder-only transformer language models from scratch in which *every* training operation is integer arithmetic: the forward pass (matrix products, RMSNorm with an integer square root, attention with an integer base-2 softmax), the backward pass (including the softmax and normalization Jacobians), and the parameter update. Weights are stored as int8 with a per-tensor power-of-two scale. There is no floating-point master copy. The optimizer is an integer, Adafactor-like rule: log2-quantized gradient magnitudes, a factored (row $\times$ column) second moment kept in integers, and an integer residual accumulator that moves a weight by whole int8 steps when it crosses a threshold. On character-level language modeling, ZeroFloat is better than a tuned AdamW baseline at small and medium sizes and slightly worse at 1B. In short runs (2000 steps, 3 seeds, 102k-character evaluation), the gap in bits/char is -0.029 (0.8M parameters), -0.045 (10.7M), -0.020 (85M) and $+0.075$ (1.0B). In longer runs (10k steps on WikiText-103 bytes), the gap is -0.007 at 10.7M and $+0.016$ at 1.0B (2.151 vs. 2.135; 3 seeds for ZeroFloat, 2 for AdamW). We found and fixed the cause of a 1B training divergence: a temperature mismatch of the base-2 softmax. We report what we tried in order to close the remaining 1B gap. Backpropagation fidelity, 8-bit matrix inputs, the int8 weight grid, the residual accumulator and seed noise are ruled out as its source. Momentum, a faster second moment, per-element second moments and per-tensor learning rates did not close it. At 1B, peak training memory is 11.2 GB vs. 20.2 GB for AdamW.

1 Introduction

Low-precision training of large language models normally keeps a high-precision "master" copy of the weights and floating-point optimizer state, and computes nonlinearities (softmax, normalization) in floating point [5, 7, 9]. Integer-only training has been demonstrated for CNNs and small vision transformers with SGD-type optimizers [2, 3, 4], and recent work removes master weights in quantized (but not integer-only) training [6]. We ask whether a language model can be trained from scratch with integer-only arithmetic throughout, with an adaptive optimizer, and how close it gets to AdamW as the model grows.

Contributions. (1) A fully integer training pipeline for decoder-only transformers, with no floating-point operation in the forward pass, backward pass, or update (Sec. 2). (2) An integer adaptive optimizer built on a residual accumulator over int8 weights, with log2 gradient quantization and a factored integer second moment. (3) Results from 0.8M to 1.0B parameters against a tuned AdamW baseline, in short and long runs, with multiple seeds (Sec. 4). (4) Diagnoses of what

breaks integer training at scale: a softmax temperature mismatch that caused 1B divergence, and silent accumulator overflow (Sec. 5). (5) A record of the experiments run to close the remaining 1B gap, separating what was ruled out from what was tried without effect (Sec. 6).

Origin of the method. The residual accumulator and the choice to spend bits on the accumulator rather than on the weight come from the author's earlier work on lattice-quantized multiplicative weights for online linear learning [1]. That work shows that the "virtual" iterate (weight plus residual) follows the unquantized update exactly, while the stored weight stays within one lattice step of it. We measure the same property in language model training (Sec. 6). The accumulator is a form of error feedback [11]; the factored second moment follows Adafactor [10].

2 Method

Number formats. Each weight tensor n is stored as $W_n^{(8)} \in [-127, 127]$ (int8). Its value on a 2^{-11} grid is $W_n = W_n^{(8)} \cdot 2^{s_n}$. The shift $s_n \in \{0, \dots, 4\}$ is chosen once at initialization so that the initial standard deviation is about 16 grid steps. Activations are fixed point with 8 fractional bits.

Matrix products. Before every matrix product, each operand is reduced to 8 bits along the contraction axis with a power-of-two scale: round-to-nearest in the forward pass, stochastic rounding in the backward pass. Products are exact integers. In our implementation they are evaluated through float64 GEMMs, whose results are exact below 2^{53}.

Integer RMSNorm. For a row $x \in \mathbb{Z}^D$: $v = \lfloor (\sum_i x_i^2 + D/2)/D \rfloor$, $r = \max(\mathrm{isqrt}(v), 8)$ by Newton iteration, $y_i = \lfloor (2^8 x_i + r/2)/r \rfloor$, output $y + (y \odot g \gg 11)$. The backward pass uses the exact integer Jacobian $\partial y = (\delta - y \, \langle \delta, y \rangle / (D \, 2^8))/r$ with the same rounding.

Integer softmax. Attention scores S are first multiplied by $1477/1024 \approx 1/\ln 2$ (one integer multiply and shift), so that the base-2 softmax has the same temperature as the usual base-e softmax. Logits are shifted by the row maximum, $u = \max - s \geq 0$. With $k = \lfloor u/2^8 \rfloor$ and fraction ρ, we use $2^{-u} \approx (1 - \rho/2)\, 2^{-k}$ in 12-bit (attention) or 16-bit (loss) fixed point, then normalize by the integer row sum. The loss gradient is the integer softmax minus the one-hot target. The MLP activation is ReLU clipped to $[0, 2]$.

Integer adaptive update. For a gradient entry g, let $e = \mathrm{round}(\log_2 |g|)$ (with a $\sqrt{2}$ midpoint). Squared magnitudes 2^{2e} are averaged over non-zero entries per row and per column and tracked by integer EMAs R_i, C_j with rate 2^{-k_v} ($\beta_2 = 1 - 2^{-k_v}$). We use $k_v = 5$ in short runs and $k_v = 7$ in long runs. The second moment is factored in the log domain (4 fractional bits),

$$\log_2 \hat{v}_{ij} = \log_2 R_i + \log_2 C_j - \log_2 \overline{R},$$

and the normalized step is $u_{ij} = \mathrm{sgn}(g_{ij})\, 2^{e_{ij} - \frac{1}{2} \log_2 \hat{v}_{ij}}$, clipped to $[2^{-10}, 2^2]$ and written in 1/16 steps. A neighbor pull moves each step towards its row mean: $u_{ij} \leftarrow u_{ij} + (\bar{u}_{i\cdot} - u_{ij})/8$. The step is added to an integer accumulator A. When $|A| \geq T$, the weight moves by $\lfloor |A|/T \rfloor$ int8 steps and A keeps the remainder. There is no momentum.

Threshold schedule. The threshold $T = \mathrm{hp} \cdot 2^{10}$ plays the role of an inverse learning rate. It follows a cosine schedule from hp_0 to $3.3\,\mathrm{hp}_0$. In long runs, a tail brake additionally multiplies T by a factor that grows linearly from 1 to 2 over the last quarter of training. At 1B, $\mathrm{hp}_0 = 13.7$ (short) and 19 (long).

Accumulator width. $|A| < T$ after every update, so A fits in int16 only while $\mathrm{hp} < 32$. Overflow is silent (the stored value wraps), so we store A in int32. Persistent state is about 5 bytes per parameter (int8 weight + int32 accumulator; the factored second moment is $O(m+n)$ per matrix), vs. 12 for AdamW. A 1-byte accumulator (stored as a fraction of T with stochastic rounding, 2 bytes/parameter in total) matched the 4-byte one on the smallest model (2.780 vs. 2.784 bits/char, 1000 steps); we have not tested it at larger sizes.

3 Experimental setup

Decoder-only transformers with RMSNorm and context 32. Sizes: S (4 layers, D=128, 0.8M parameters), M (6, 384, 10.7M), L (12, 768, 85M), XL (24, 1024, 302M), G1 (20, 2048, 1.0B).

Short runs. Character-level Tiny Shakespeare ($V = 65$), 2000 steps $\times$ 512 tokens (1M tokens).

Long runs. WikiText-103 at byte level (first 150 MB), 10k steps $\times$ 512 tokens (5.1M tokens).

Baseline. The AdamW baseline uses the same architecture in fp32, with the same clipped ReLU and the same initialization. Its settings are warmup 100, $\beta_1 = 0.9$, $\beta_2 = 0.95$, gradient clipping 1.0, cosine decay and no weight decay. In short runs, the learning rate was chosen from a grid whose optimum is interior at every size. In long runs at M, we used a grid of three learning rates. At 1B, we used $4 \cdot 10^{-5}$, scaled from the short-run optimum, without a long-run grid.

Evaluation. The metric is bits per character on 100 fixed validation pieces of 1024 characters (102,400 characters, the same pieces for every method; lower is better). Hardware: one NVIDIA A100-40GB per run (Google Colab).

4 Results

Model	Params	seeds	ZeroFloat	AdamW	gap
S	0.8M	3	2.627	2.656	−0.029
M	10.7M	3	2.557	2.602	−0.045
L	85M	3	2.522	2.542	−0.020
G1	1.0B	3	2.610	2.535	+0.075

Table 1: Short runs (2000 steps, Tiny Shakespeare), bits/char, mean over seeds. Without the softmax temperature correction, the gaps were −0.049, −0.060, −0.001, +0.032 (XL, 302M) and +0.063.

ZeroFloat is better than AdamW at small and medium sizes and worse at 1B, in both settings (Tables 1, 2). The gap moves against ZeroFloat as the model grows. At 1B, the ZeroFloat seeds

Model	ZeroFloat	AdamW	gap
M (10.7M)	2.154 (3 seeds)	2.161 (best of 3 lr, 1 seed)	−0.007
G1 (1.0B)	2.149 / 2.155 / 2.149 = **2.151**	2.135 / 2.136 = **2.135**	+0.016

Table 2: Long runs (10k steps, WikiText-103 bytes), bits/char. At 1B, without the tail brake, ZeroFloat gives 2.170 (3 seeds, ±0.003).

agree within ±0.003, and the two AdamW seeds differ by 0.0012, so the 1B gap is larger than seed noise.

Memory and speed. Peak training memory at 1B ($B = 16$, $T = 32$) is 11.2 GB for ZeroFloat and 20.2 GB for AdamW (fp32). Our reference implementation evaluates exact integer products through float64 GEMMs; at 1B it takes 325 ms/step vs. 261 ms for AdamW. An int8 tensor-core path took 164 ms/step at 1B (1.95× faster than the reference) with matching loss over 200 steps. We have not verified that path to be bit-identical on GPU, so we do not claim a speed advantage.

5 What breaks integer training at scale

Softmax temperature. Without the $1/\ln 2$ correction, the base-2 attention softmax is sharper than the base-e softmax it replaces. In 1B long runs, this led to divergence after step 5000 (2.378 at step 5000, 3.36 at the end, vs. AdamW 2.135). The query weights and the first RMSNorm gain grew until they saturated at ±127. Rescaling saturated tensors alone did not prevent the divergence, so saturation was a symptom. With the correction, no 1B run diverged, and the query weights stayed smaller ($|W|$ 19.6 vs. 26.8).

Silent accumulator overflow. With an int16 accumulator, 1B short runs at $\mathrm{hp}_0 = 13.7$ and 20 overflowed after 55% and 34% of training, giving 2.750 and 3.016 bits/char instead of 2.522 and 2.636 with int32. Integer training at scale needs explicit range analysis of every stored integer, not only of the matrix products.

Earlier fixes. With a single global weight scale and a slow, row-only second moment, the short-run gap grew quickly with size (S +0.02, M +0.07, L +0.22). Per-tensor power-of-two weight scales and the factored row×column second moment removed most of this growth.

6 The remaining gap at 1B

We ran a series of experiments to find the source of the +0.016 gap at 1B and to close it. The results below are diagnostic: they say what the gap is *not* under our conditions, and which changes did not help. They do not identify its source. Most of the 1B interventions were run with one seed, and we tested each change separately, so combinations remain untested.

Ruled out as the source.

- *Backpropagation.* On identical weights and data, integer gradients match a float64 twin model with the same mathematics (autograd): cosine similarity 0.992–1.000 per tensor group, sign agreement 99.9–100%. With 8-bit matrix inputs, cosine is 0.981–0.999.

- *8-bit matrix inputs.* At 1B (short run, one seed), a full-precision backward pass gave 2.601 vs. 2.603.
- *The int8 weight grid.* AdamW constrained to the same per-tensor int8 grid (fp32 master, straight-through gradient) gave 2.139 and 2.136 in 1B long runs, vs. 2.135 and 2.136 unconstrained (+0.002 on average).
- *The residual accumulator.* Fed with AdamW's own gradient stream (S, 1500 steps), our full rule's displacement summed over 500 steps has cosine 0.86 to AdamW's. Swapping only the accumulator and threshold into AdamW gives cosine 0.994–1.000 at that horizon. This is the property predicted by the virtual-iterate argument of [1].
- *Seed noise.* See Sec. 4.

Update rule versus AdamW. With AdamW's components replaced one at a time on the same gradient stream (S), the largest long-horizon deviations come from the slower second moment (cosine 0.924 at 500 steps) and the factored second moment (0.973). Removing momentum costs little at that horizon (0.975), although it costs much over a single step (0.43). Log2 gradient quantization (0.996), 8-bit gradients (0.997; 1.000 with separate row and column scales) and the neighbor pull (1.000) cost almost nothing. Storing a per-element second moment in 8 bits costs nothing measurable (1.000).

In float 1B long runs with AdamW's setup, removing momentum gave 2.223 (+0.089), and additionally slowing the second moment to $\beta_2 = 0.992$ gave 2.209. ZeroFloat (2.151) is therefore 0.06–0.07 bits/char better than momentum-free float AdamW at 1B. A row-only second moment gave 2.500, which confirms that the column factor is necessary.

Tried, did not close the gap (1B, long runs).

Change	seeds	result	reference
Momentum on raw gradients from step 7500	3	2.1513	2.1511
Momentum + faster 2nd moment ($\beta_2 \approx 0.94$) from 7500	3	2.1549	2.1511
Per-element second moment (last quarter)	1	2.1530	2.1549
Finer gradient magnitudes (4 fractional bits)	1	2.1513	2.1489
Sign-consistency noise brake	1	2.1842	2.1494
Larger thresholds: gains, biases, embeddings ($\times 4/\times 16$)	1	2.1619 / 2.1660	2.1494
Uniform effective lr across tensors (as AdamW)	1	diverged (3.62)	2.1494
Stronger tail brake ($\times 10$)	2	2.206 / 2.199	2.174 / 2.167
AdamW-shaped step schedule (3 floors)	1	2.199 / 2.199 / 2.181	2.1494

In every 1B run, ZeroFloat leads AdamW at step 5000 (2.29–2.31 vs. 2.35 on a small evaluation set) and then stalls until step 7500, while AdamW keeps improving. Matching AdamW's effective step schedule did not remove this stall, and decaying the step faster made the final result worse. At the medium size (M, long runs), momentum on raw gradients from the start was also worse than no momentum (2.20 vs. 2.15, best of four thresholds).

What remains. The update rule as a whole reaches its own balance. It has no momentum and a slow second moment; the accumulator and threshold filter gradient noise in place of momentum. At small and medium sizes this balance beats AdamW's balance (momentum and a fast second moment); at 1B it is slightly worse. Changing individual components in the last quarter of training,

making the rule's per-step direction more similar to AdamW's, or matching AdamW's step schedule did not close the gap. We have not tested a full integer copy of AdamW trained from the start at 1B, threshold and schedule tuning at 1B, or longer training with more tokens.

7 Limitations

- *Small training budget.* The 1B model sees 5.1M tokens, far fewer than compute-optimal training (about 20B tokens). The results describe early training and may change with more data.
- *One task and one metric.* Character-level language modeling on two corpora, measured in bits/char. There are no word-level or downstream evaluations.
- *Scale.* The largest model is 1.0B. Long runs exist only at two sizes, so extrapolation to larger models is not supported.
- *Tuning at 1B.* Neither the AdamW learning rate nor the ZeroFloat threshold was tuned on a grid in 1B long runs. Both were scaled from smaller models. The tail-brake factor was chosen on 1B seeds and then confirmed on all three, which is a mild selection effect.
- *Speed.* No speed advantage is claimed (Sec. 4).
- *Evaluation in float.* Evaluation (bits/char) and text sampling convert the final logits to floating point. This is a measurement step only and is not part of training.

8 Related work

NITI [2] and NITRO-D train CNNs with integer-only arithmetic and SGD. Ghaffari et al. [3] use integer forward, backward and SGD for vision models, including ViT fine-tuning, with floating-point softmax. The concurrent work of Shalby et al. [4] stores all persistent tensors as INT8, with integer softmax and normalization and SGD with momentum, on ResNets and ViTs; it lists language models as future work. Jetfire [5] pretrains transformers with an INT8 data flow but floating-point nonlinearities and optimizer. ECO [6] removes master weights in quantized training. Integer fine-tuning of BERT [8] keeps the update in FP32. Adafactor [10] introduced the factored second moment, and error feedback [11] the residual accumulation of compression error, both of which our integer rule uses. 8-bit optimizers [12] quantize optimizer state but keep floating-point weights and arithmetic.

9 Conclusion

A language model can be trained from scratch with integer arithmetic only, with no floating-point master weights and an adaptive integer optimizer, up to 1B parameters. ZeroFloat is better than AdamW at small and medium sizes and 0.016 bits/char (0.75%) worse at 1B, with about half the peak training memory. We ruled out the integer arithmetic itself (backpropagation, 8-bit inputs, the int8 weight grid, the accumulator) as the source of the 1B gap. What remains is a difference between the two optimizers' behavior at scale that we have not been able to remove.

Acknowledgments. An AI assistant was used as a tool for writing code and running experiments. The method, ideas and analysis are the author's.

License. The method description is public. The reference implementation is licensed for personal and non-commercial use; commercial use requires a paid license from the author.

Provenance. Results and code were timestamped with OpenTimestamps (Bitcoin):
`v2026-10-16-300m` (sha256 b942f9e1…9b81) and
`v2026-10-18-1b-esit` (sha256 e73cffaa…9f53c6).

Changes from version 1. Version 1 claimed parity with AdamW up to 1B based on a single 1B seed and a 3-piece evaluation (gap +0.008). With a 100-piece evaluation and three seeds, the short-run 1B gap is +0.063 (+0.075 with the softmax correction). Version 2 corrects the title and abstract and adds long runs, the softmax temperature fix, the neighbor pull, the tail brake, and the analysis of the remaining gap.

References

[1] M. Ökmen. Lattice-quantized multiplicative weights: how many bits does an online linear learner need? Technical report, Zenodo, 2026. doi:10.5281/zenodo.23568755.

[2] M. Wang et al. NITI: Training integer neural networks using integer-only arithmetic. arXiv:2009.13108.

[3] A. Ghaffari et al. Is integer arithmetic enough for deep learning training? NeurIPS 2022. arXiv:2207.08822.

[4] H. H. Y. Shalby, F. Pittorino, M. Roveri. Memory-efficient integer-only training of deep neural networks. arXiv:2610.16298, 2026.

[5] H. Xi et al. Jetfire: Efficient and accurate transformer pretraining with INT8 data flow and per-block quantization. arXiv:2403.12422.

[6] M. Nikdan, A. Zandieh, D. Alistarh, V. Mirrokni. ECO: Quantized training without full-precision master weights. ICML 2026. arXiv:2601.22101.

[7] H. Xi et al. Training transformers with 4-bit integers. NeurIPS 2023. arXiv:2306.11987.

[8] Towards fine-tuning pre-trained language models with integer forward and backward propagation. Findings of EACL 2023. arXiv:2209.09815.

[9] Low-precision training of large language models: methods, challenges, and opportunities. arXiv:2505.01043.

[10] N. Shazeer, M. Stern. Adafactor: Adaptive learning rates with sublinear memory cost. ICML 2018.

[11] S. P. Karimireddy, Q. Rebjock, S. U. Stich, M. Jaggi. Error feedback fixes SignSGD and other gradient compression schemes. ICML 2019.

[12] T. Dettmers, M. Lewis, S. Shleifer, L. Zettlemoyer. 8-bit optimizers via block-wise quantization. ICLR 2022.